Ain Shams University

Faculty of Engineering
Electronics and Communications Engineering Department

Graduation Project Thesis — Part II

Machine Learning Techniques for PAPR Reduction in Multicarrier Communication Systems

Project Team

Nour-Eldin Mohamed Mostafa
Youssef Samir Sleem Ahmed
Muhammad Mahmoud Muhammad
Reem Nader Saeed
Basel Mohamed Ramadan
Ahmed Walid Hassan
Yosef Ahmed Mohamed

Supervisor

Dr. Michael Ibrahim

A thesis submitted in partial fulfillment of the requirements for the degree of Bachelor of Engineering

Academic Year: 2025/2026

Declaration of Authorship

We, Nour-Eldin Mohamed Mostafa, Youssef Samir Sleem Ahmed, Muhammad Mahmoud Muhammad, Reem Nader Saeed, Basel Mohamed Ramadan, Ahmed Walid Hassan, Yosef Ahmed Mohamed, declare that this thesis titled “Machine Learning Techniques for PAPR Reduction in Multicarrier Communication Systems” and the work presented in it are our own. We confirm that this material, submitted for assessment on the programme leading to the award of Bachelor of Engineering, is entirely our own work. We have exercised reasonable care to ensure that the work is original and does not, to the best of our knowledge, breach any law of copyright; it has not been taken from the work of others except to the extent that such work has been cited and acknowledged within the text.

Signed:

Signed:

Signed:

Signed:

Signed:

Signed:

Signed:

Date:

Abstract

Orthogonal Frequency-Division Multiplexing (OFDM) is the dominant waveform for modern high-speed wireless communication systems — including Wi-Fi, 4G LTE, and 5G NR — owing to its robustness against multipath fading and high spectral efficiency. Its principal drawback is a high Peak-to-Average Power Ratio (PAPR), which forces the High-Power Amplifier (HPA) at the transmitter to operate in its non-linear region, causing in-band distortion and severe power inefficiency.

This document is **Part II** of a two-semester graduation project. Part I presented a comprehensive simulation and analysis of classical PAPR reduction techniques — Amplitude Clipping and Filtering, Tone Reservation, Selected Mapping (SLM), and Partial Transmit Sequence (PTS) — establishing both their capabilities and their fundamental limitations: high computational complexity, mandatory side information, and reliance on exhaustive candidate search.

The present work applies **machine learning** to overcome those limitations. The central contribution is a detailed study of **PRNet** (Kim et al., 2017), the first end-to-end deep-learning system for joint PAPR and BER optimisation in OFDM. PRNet models the entire transmitter–channel–receiver chain as a deep autoencoder: a DNN encoder learns a data-dependent constellation shaping that suppresses time-domain peaks, while a jointly trained DNN decoder recovers the transmitted bits without requiring any side information at the receiver. The document derives the full system model, joint (MSE + PAPR) loss function, two-stage training procedure, and reproduces the BER-vs.-SNR and CCDF simulation results from the original paper — including a critical audit of the peak-amplitude proxy used in the published CCDF curve.

Chapter 1 provides a self-contained, mathematically complete refresher of OFDM, PAPR, and the SLM algorithm, establishing the notation and motivation that connect Part I to Part II. Chapter 2 develops PRNet in full detail, from autoencoder architecture and loss functions to training dynamics and real-world deployment considerations. Further ML-enhanced PAPR reduction methods will be added in subsequent chapters as the project progresses.

Acknowledgements

This project represents not only two semesters of dedicated engineering work, but the culmination of our undergraduate journey at Ain Shams University. It would not have been possible without the guidance and support of many individuals.

First and foremost, we express our deepest gratitude to our supervisor, Dr. Michael Ibrahim, for their patience, technical insight, and for steering us in the right direction whenever we encountered a challenge — whether mathematical, computational, or conceptual.

We are grateful to the faculty and staff at the Electronics and Communications Engineering Department for providing the resources and foundational knowledge required to tackle this research. A special thanks to all the professors who challenged us to think as engineers, not merely as students.

To our families: thank you for being our anchor. Your unwavering support, encouragement, and understanding during long nights and stressful deadlines meant more than words can express. We hope we have made you proud.

Finally, to our friends and fellow students — thank you for the shared study sessions, the laughter, and the moral support that carried us through.

Contents

List of Figures

List of Tables

Nomenclature

Acronyms

Acronym	Meaning
AE	Autoencoder
ANN	Artificial Neural Network
AWGN	Additive White Gaussian Noise
BatchNorm	Batch Normalization
BER	Bit Error Rate
CCDF	Complementary Cumulative Distribution Function
CP	Cyclic Prefix
CR	Clipping Ratio
DFT	Discrete Fourier Transform
DL	Deep Learning
DNN	Deep Neural Network
FC	Fully Connected
FDM	Frequency-Division Multiplexing
FFT	Fast Fourier Transform
HPA	High-Power Amplifier
IBO	Input Back-Off
IDFT	Inverse Discrete Fourier Transform
IFFT	Inverse Fast Fourier Transform
ISI	Inter-Symbol Interference

Acronym	**Meaning**
LTE	Long-Term Evolution
ML	Machine Learning
MSE	Mean Squared Error
NN	Neural Network
NR	New Radio (5G)
OFDM	Orthogonal Frequency-Division Multiplexing
PAPR	Peak-to-Average Power Ratio
PRNet	PAPR Reduction Network
PTS	Partial Transmit Sequence
QAM	Quadrature Amplitude Modulation
QPSK	Quadrature Phase Shift Keying
ReLU	Rectified Linear Unit
SGD	Stochastic Gradient Descent
SI	Side Information
SLM	Selected Mapping
SNR	Signal-to-Noise Ratio
TI	Tone Injection
TR	Tone Reservation

Mathematical Symbols

Symbol	Definition
N	Number of OFDM subcarriers
k	Subcarrier index ($k = 0, 1, \ldots, N-1$)
n	Time-domain sample index ($n = 0, 1, \ldots, N-1$)
U	Number of SLM candidate phase sequences
u	Index of a specific SLM candidate ($u = 1, \ldots, U$)
$\hat{u}$	Index of the selected (best) SLM candidate
M	Constellation size (number of points per subcarrier)
L	Oversampling factor
$\boldsymbol{X}$	Frequency-domain OFDM data symbol vector
X_k	Complex symbol on the k-th subcarrier
$x(n)$	Time-domain OFDM signal sample at index n
$\boldsymbol{x}$	Time-domain OFDM signal vector
$\boldsymbol{n}$	AWGN noise vector
$\boldsymbol{\psi}_u$	u-th SLM phase sequence vector
$\psi_{u,k}$	Phase factor for subcarrier k in sequence u
$\varphi_{u,k}$	Phase angle of $\psi_{u,k}$
$\boldsymbol{W}$	Weight matrix of a neural network layer
$\boldsymbol{b}$	Bias vector of a neural network layer
$\mathcal{L}$	Loss function
A	Clipping threshold

Symbol	Definition
CR	Clipping ratio
λ	PAPR loss weight / trade-off parameter (PRNet)
η	Training corruption level, noise-to-signal ratio (PRNet)
θ_f	Encoder network trainable parameters
θ_g	Decoder network trainable parameters
$\boldsymbol{r}$	PRNet input: one-hot encoded QPSK symbol vector
$\hat{\boldsymbol{r}}$	PRNet output: reconstructed symbol scores
$\boldsymbol{X}_{\text{enc}}$	Encoded complex subcarrier symbols (encoder output)
$f(\cdot;\theta_f)$	Encoder neural network function
$g(\cdot;\theta_g)$	Decoder neural network function
P_{sat}	HPA saturation power
P_{avg}	Average signal power
N_{cp}	Cyclic prefix length
Δf	Subcarrier spacing
T_s	Useful OFDM symbol duration
$\odot$	Element-wise (Hadamard) product
$(\cdot)^*$	Complex conjugate
$(\cdot)^T$	Transpose
$\|\cdot\|_2$	Euclidean (ℓ_2) norm
$E[\cdot]$	Expected value
$\Re(\cdot)$	Real part of a complex number
$\Im(\cdot)$	Imaginary part of a complex number

CHAPTER 1

OFDM & SLM Refresher

Chapter Motivation

You already studied Orthogonal Frequency-Division Multiplexing (OFDM) and the Selected Mapping (SLM) algorithm during Graduation Project 1. This chapter is *not* a tutorial from scratch—it is a precise, mathematically complete refresher that (i) consolidates the key concepts, (ii) establishes the exact notation used throughout this document, and (iii) arrives at a crystal-clear statement of SLM's three core limitations, which directly motivate the PRNet approach studied in the following chapter.

1.1 OFDM in a Nutshell

Orthogonal Frequency-Division Multiplexing divides a wideband channel into N narrowband, orthogonal subcarriers. Each subcarrier carries one modulated symbol from a constellation such as QPSK or M-QAM. The key enabler of OFDM is the efficient realisation of modulation and demodulation via the Inverse Fast Fourier Transform (IFFT) and Fast Fourier Transform (FFT), respectively.

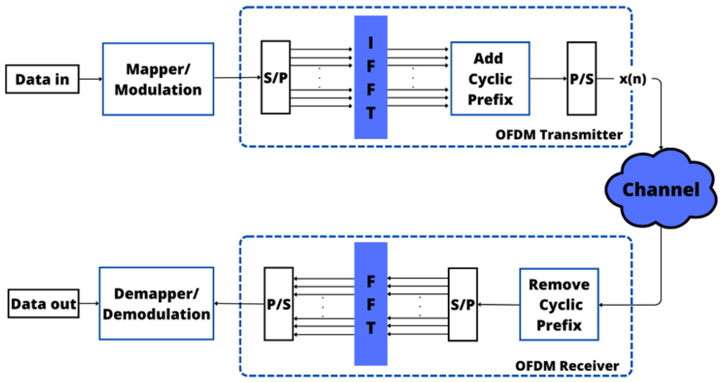

FIGURE 1.1: Block diagram of a baseband OFDM transmitter and receiver. The transmitter maps data onto N subcarriers via an N-point IFFT, adds a cyclic prefix (CP), and transmits the time-domain signal. The receiver removes the CP and applies an N-point FFT to recover the frequency-domain symbols.

1.1.1 Subcarrier Structure and the IFFT/FFT Relationship

Let the frequency-domain data symbol vector after constellation mapping be

$$\boldsymbol{X} = \left[X_0,\ X_1,\ \ldots,\ X_{N-1}\right]^T,$$

where X_k is the complex symbol transmitted on the k-th subcarrier and N is the total number of subcarriers. The time-domain OFDM signal is obtained by applying the N-point IFFT:

1.1

$$x(n) = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X_k\, e^{j2\pi kn/N}, \qquad n = 0,\, 1,\, \ldots,\, N-1.$$

Key Insight

The $1/\sqrt{N}$ normalization is a matter of convention. Some references use $1/N$ instead. Throughout this document we adopt the $1/\sqrt{N}$ form so that the signal energy is preserved between the frequency and time domains (Parseval's theorem).

At the receiver, the frequency-domain symbols are recovered by the complementary FFT operation:

1.2

$$X_k = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} x(n)\, e^{-j2\pi kn/N}, \qquad k = 0,\, 1,\, \ldots,\, N-1.$$

1.1.2 Cyclic Prefix

Before transmission, a **Cyclic Prefix (CP)** of length N_{cp} samples is prepended to each OFDM symbol by copying the last N_{cp} samples of the IFFT output to the front. The CP serves two critical purposes:

1. **Inter-Symbol Interference (ISI) elimination:** As long as N_{cp} exceeds the maximum channel delay spread, the CP absorbs all multipath echoes from the previous symbol.

2. **Circular convolution:** The CP converts the linear convolution with the channel into a circular convolution, enabling single-tap equalization in the frequency domain via the FFT.

1.1.3 Spectrum Efficiency: FDM vs. OFDM

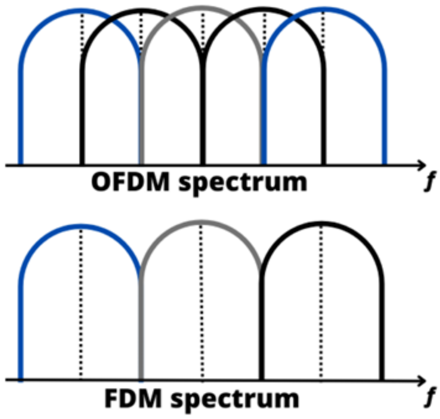

FIGURE 1.2: Ideal spectrum comparison between conventional Frequency-Division Multiplexing (FDM) and OFDM. In FDM, guard bands between channels waste spectrum. In OFDM, orthogonal subcarriers overlap without mutual interference, achieving significantly higher spectral efficiency.

The spectrum comparison above illustrates why OFDM is spectrally superior: the subcarriers are spaced at exactly $\Delta f = 1/T_s$ (where T_s is the useful symbol duration), ensuring orthogonality. At every subcarrier's centre frequency, all other subcarriers have a spectral null, so they contribute zero interference.

1.2 PAPR: Definition and Significance

1.2.1 Formal Definition

Because the time-domain OFDM signal — the sum of N independently modulated complex exponentials — can occasionally see all subcarrier contributions align constructively, large amplitude peaks appear. The severity of this peaking is quantified by the **Peak-to-Average Power Ratio (PAPR)**:

1.3

$$\mathrm{PAPR}(\boldsymbol{x}) = \frac{\max\limits_{0 \le n \le N-1} \left|x(n)\right|^2}{E\left[\left|x(n)\right|^2\right]},$$

where the numerator is the peak instantaneous power and the denominator is the average (expected) power of the OFDM symbol.

Numerical Feel for PAPR

For a 128-subcarrier OFDM system with QPSK modulation, the theoretical worst-case PAPR is $10\log_{10}(128) \approx 21.1$ dB—this occurs when all 128 subcarriers happen to align in phase at some instant n. In practice, such perfect alignment is extremely rare; the typical PAPR at a CCDF of 10^{-2} is around 10–12 dB. Even so, this is far too high for efficient power amplifier operation.

1.2.2 CCDF as the Evaluation Metric

Since PAPR is a random variable (it changes from one OFDM symbol to the next), we characterise its statistical behaviour using the **Complementary Cumulative Distribution Function (CCDF)**:

$$\mathrm{CCDF}_{\mathrm{PAPR}} = \Pr\big(\mathrm{PAPR} > \mathrm{PAPR}_0\big),$$

where PAPR_0 is a given threshold. The CCDF tells us the probability that any randomly generated OFDM symbol will exceed a certain PAPR level. A good PAPR reduction technique shifts the CCDF curve to the *left*, meaning that high-PAPR events become much rarer.

For N subcarriers with independent, identically distributed symbols and Nyquist-rate sampling, the CCDF can be approximated as:

1.4

$$\Pr\big(\mathrm{PAPR} \geq \mathrm{PAPR}_0\big) \approx 1 - \big(1 - e^{-\mathrm{PAPR}_0}\big)^{N}.$$

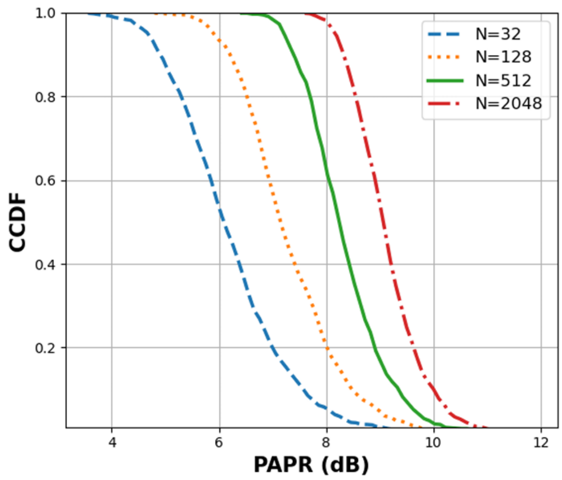

FIGURE 1.3: CCDF of PAPR for different numbers of subcarriers ($N = 32, 128, 512, 2048$) with 16-QAM modulation. As N increases, the PAPR distribution shifts to the right, confirming that more subcarriers lead to higher PAPR.

1.2.3 The HPA Nonlinearity Problem

Why does high PAPR matter in practice? The answer lies in the **High-Power Amplifier (HPA)** at the transmitter. An ideal HPA would amplify any input linearly, regardless of amplitude. In reality, every power amplifier has a *saturation region*: when the input signal exceeds a certain amplitude, the output clips or compresses, introducing severe nonlinear distortion.

To avoid entering this nonlinear zone, the HPA must operate with sufficient **Input Back-Off (IBO)**:

$$\mathrm{IBO}\ (\mathrm{dB}) = 10\log_{10}\left(\frac{P_{\text{sat}}}{P_{\text{avg}}}\right),$$

where P_{sat} is the amplifier's saturation power and P_{avg} is the average input signal power. For an OFDM signal with PAPR $=$ 12 dB, the IBO must be at least 12 dB, meaning the amplifier operates far below its maximum capacity most of the time. This directly costs *power efficiency* and *hardware complexity* in real systems.

Key Insight

Reducing PAPR by even 3–4 dB translates to a significant improvement in HPA efficiency, reduced heat dissipation, longer battery life in mobile devices, and lower deployment costs. This is why PAPR reduction is one of the most active research areas in OFDM system design.

1.3 Classical PAPR Reduction Techniques — A Brief Survey

Before diving into SLM, it is helpful to briefly survey the landscape of classical PAPR reduction techniques, as many of them appear as baselines in the ML-enhanced approaches studied later. These can be broadly classified into **signal distortion** techniques and **signal scrambling** (or probabilistic) methods.

1.3.1 Signal Distortion Techniques

Clipping and Filtering. The simplest approach: any signal sample exceeding a threshold A is clipped to that threshold. Filtering is then applied to suppress out-of-band spectral regrowth. The clipped signal is:

$$x'(n) = \begin{cases} x(n), & |x(n)| \leq A, \\ \dfrac{A}{|x(n)|}\, x(n), & |x(n)| > A, \end{cases}$$

where $A = \mathrm{CR} \cdot \sqrt{P_{\text{avg}}}$ and CR is the clipping ratio.

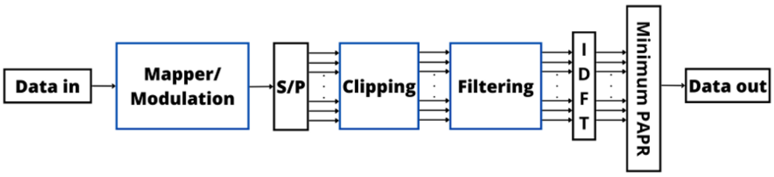

FIGURE 1.4: Block diagram of an OFDM transmitter with clipping and filtering. The signal is clipped in the time domain, then filtered in the frequency domain to remove out-of-band distortion.

Pros: Simple, no side information needed. **Cons:** Introduces in-band distortion (increased BER) and out-of-band radiation.

Active Constellation Extension (ACE). Extends outer constellation points away from the decision boundaries to reduce peak power, without affecting the BER. No side information is required.

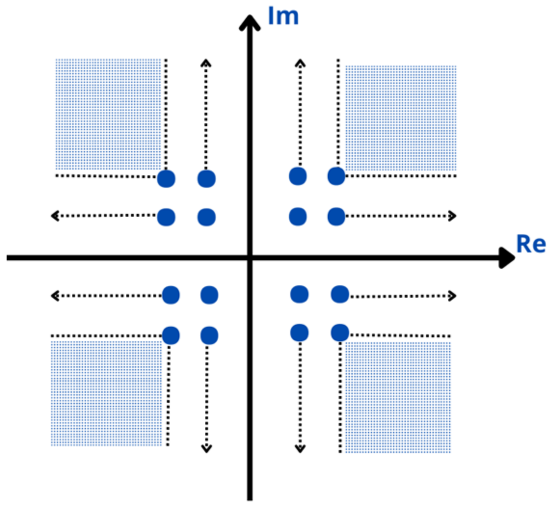

FIGURE 1.5: Active Constellation Extension for 16-QAM. Outer constellation points are extended into "feasible regions" (shaded), reducing the PAPR without crossing decision boundaries.

1.3.2 Signal Scrambling Methods

These methods alter the signal representation *without distortion* to find a low-PAPR version.

Partial Transmit Sequence (PTS). The input data block is partitioned into V disjoint sub-blocks, each transformed by an IFFT independently. The sub-blocks are then weighted by phase factors $b_v \in \{e^{j\phi} : \phi \in \Phi\}$ and summed. The phase combination yielding the lowest PAPR is selected for transmission.

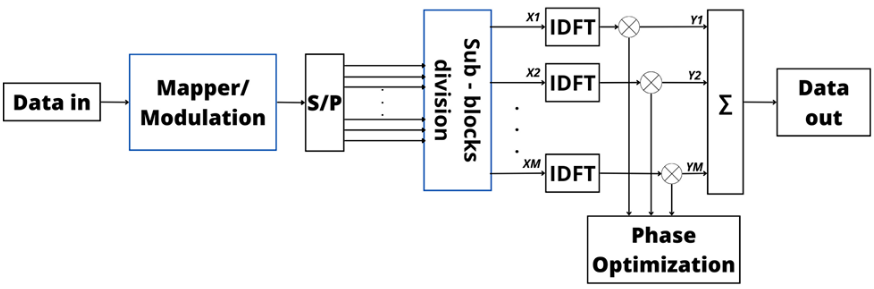

FIGURE 1.6: Block diagram of an OFDM transmitter with Partial Transmit Sequence (PTS). The data is split into V sub-blocks, each weighted by a phase factor after the IDFT. The combination with the lowest PAPR is transmitted.

Tone Reservation (TR) and Tone Injection (TI). TR reserves a set of subcarriers exclusively for peak-cancelling signals, while TI adds carefully designed tones to the data subcarriers. Both reduce PAPR without distorting data symbols, but at the cost of reduced spectral efficiency (TR) or increased average power (TI).

Interleaving. Multiple interleaving patterns are applied to the data, and the pattern producing the lowest PAPR is selected, similar to SLM but using permutation rather than phase rotation.

The comparison table below summarizes the key trade-offs.

TABLE 1.1: Comparison of classical PAPR reduction techniques. Data adapted

Technique	BER Impact	Complexity	Side Info	Distortion
PTS	None	High	Yes	No
SLM	None	High	Yes	No
Interleaving	None	Low	Yes	No
Clipping	Yes	Low	No	Yes
TI	None	High	No	No
TR	None	High	Yes	No
ACE	None	Moderate	No	No

1.4 The SLM Algorithm — Formal Review

Among all classical PAPR reduction techniques, **Selected Mapping (SLM)** stands out for its combination of distortion-free operation and applicability to any modulation format and any number of subcarriers. This section gives a complete, self-contained review.

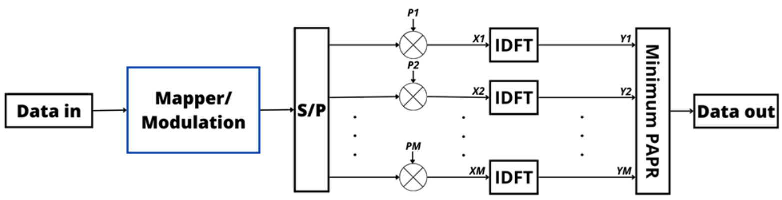

FIGURE 1.7: Block diagram of an OFDM transmitter with Selected Mapping (SLM). U different phase sequences are applied to the frequency-domain data *before* the IDFT. The candidate signal with the lowest PAPR is selected for transmission.

1.4.1 Phase Sequence Generation and Design

The core idea of SLM is to generate U alternative representations of the same data block by multiplying the frequency-domain symbols with U different **phase sequences**. Let the u-th phase sequence be:

$$\boldsymbol{\psi}_u = \left[\psi_{u,0},\ \psi_{u,1},\ \ldots,\ \psi_{u,N-1}\right]^T, \qquad u = 1,\, 2,\, \ldots,\, U,$$

where each element is a unit-magnitude complex number:

$$\psi_{u,k} = e^{j\varphi_{u,k}}, \qquad \varphi_{u,k} \in [0,\ 2\pi).$$

In the simplest case, $\varphi_{u,k}$ is drawn uniformly at random from $[0, 2\pi)$. In practice, the phase alphabet is often restricted to a finite set such as $\{+1,\ -1,\ +j,\ -j\}$ (i.e., $\varphi_{u,k} \in \{0,\ \pi/2,\ \pi,\ 3\pi/2\}$) to reduce implementation complexity.

Key Insight

One of the U sequences is always set to the all-ones vector $\boldsymbol{\psi}_1 = [1,\ 1,\ \ldots,\ 1]^T$, so that one candidate is the unmodified original signal. This guarantees that SLM can never *increase* the PAPR beyond the original value.

1.4.2 Candidate Signal Generation

Each phase sequence is applied element-wise (Hadamard product) to the frequency-domain data vector:

$$\boldsymbol{X}_u = \boldsymbol{X} \odot \boldsymbol{\psi}_u, \qquad u = 1,\ 2,\ \ldots,\ U.$$

The corresponding time-domain candidate signals are then obtained via the IFFT:

$$\boldsymbol{x}_u = \mathrm{IFFT}\{\boldsymbol{X}_u\} = \mathrm{IFFT}\{\boldsymbol{X} \odot \boldsymbol{\psi}_u\}, \qquad u = 1,\ 2,\ \ldots,\ U.$$

1.4.3 PAPR Evaluation and Candidate Selection

The PAPR of each candidate is computed, and the one with the *minimum* PAPR is selected for transmission:

1.5

$$\hat{u} = \arg\min_{1 \le u \le U} \ \mathrm{PAPR}(\boldsymbol{x}_u).$$

The transmitted signal is then $\boldsymbol{x}_{\hat{u}}$.

1.4.4 Side Information (SI)

The receiver needs to know *which* phase sequence was used, so it can reverse the phase rotation and recover the original data:

$$\hat{\boldsymbol{X}} = \boldsymbol{X}_{\hat{u}} \odot \boldsymbol{\psi}_{\hat{u}}^{*},$$

where $(\cdot)^*$ denotes the element-wise complex conjugate (since $|\psi_{u,k}| = 1$, dividing by $\psi_{u,k}$ is the same as multiplying by $\psi_{u,k}^*$).

The index $\hat{u}$ is the **side information (SI)**, which requires $\lceil \log_2 U \rceil$ bits per OFDM symbol. For example, $U = 16$ candidates require 4 SI bits.

Common Misconception

"Side information is a minor overhead."

While $\lceil \log_2 U \rceil$ bits may seem negligible, these bits must be received *error-free* for the receiver to undo the correct phase rotation. If even one SI bit is received in error, the *entire* OFDM symbol is incorrectly de-rotated, causing a burst of bit errors. Protecting SI typically requires additional channel coding or embedding schemes, which add further complexity and overhead.

1.5 Worked Numerical Recap

SLM with $N = 8$, $U = 4$

Consider a toy OFDM system with $N = 8$ subcarriers and QPSK modulation. The frequency-domain data vector is:

$$\boldsymbol{X} = [1+j,\ -1+j,\ 1-j,\ -1-j,\ 1+j,\ -1+j,\ 1-j,\ -1-j]^T.$$

We use $U = 4$ phase sequences with alphabet $\{+1, -1\}$:

$$\begin{aligned}
\boldsymbol{\psi}_1 &= [+1,\ +1,\ +1,\ +1,\ +1,\ +1,\ +1,\ +1]^T && \text{(identity)},\\
\boldsymbol{\psi}_2 &= [+1,\ -1,\ +1,\ -1,\ +1,\ -1,\ +1,\ -1]^T,\\
\boldsymbol{\psi}_3 &= [+1,\ +1,\ -1,\ -1,\ +1,\ +1,\ -1,\ -1]^T,\\
\boldsymbol{\psi}_4 &= [+1,\ -1,\ -1,\ +1,\ +1,\ -1,\ -1,\ +1]^T.
\end{aligned}$$

Step 1: Generate candidates. For each u, compute $\boldsymbol{X}_u = \boldsymbol{X} \odot \boldsymbol{\psi}_u$, then $\boldsymbol{x}_u = \mathrm{IFFT}\{\boldsymbol{X}_u\}$.
Step 2: Compute PAPR. Evaluate $\mathrm{PAPR}(\boldsymbol{x}_u)$ for $u = 1, \ldots, 4$.
Step 3: Select. Suppose the PAPR values are 9.2 dB, 6.1 dB, 7.8 dB, and 5.3 dB for $u = 1, 2, 3, 4$ respectively. We select $\hat{u} = 4$ (lowest PAPR = 5.3 dB).
Step 4: Transmit. Send $\boldsymbol{x}_4$ along with 2 bits of SI ($\hat{u} = 4 \to$ binary "11").
Step 5: Recover. The receiver uses the SI to look up $\boldsymbol{\psi}_4$ and computes $\hat{\boldsymbol{X}} = \boldsymbol{X}_4 \odot \boldsymbol{\psi}_4^*$ to recover the original $\boldsymbol{X}$.
In this example, SLM reduced the PAPR from 9.2 dB (original) to 5.3 dB—a gain of 3.9 dB—at the cost of 4 IFFT operations and 2 SI bits.

1.6 SLM Variants Overview

Several variants of classical SLM appear as baselines or building blocks in the ML-enhanced approaches studied in later chapters:

- **Conventional SLM**: Uses random or pseudo-random phase sequences with a finite phase alphabet. The PAPR reduction increases with U but with diminishing returns, as the CCDF plot below illustrates.
- **Modified SLM**: Reduces the number of IFFT operations required by exploiting structural relationships between phase sequences.

- **Extended SLM (ESLM)**: Extends SLM by allowing phase factors to take *any* value in $[0, 2\pi)$ (rather than a discrete alphabet), and determines them adaptively through neural network training.
- **GA-Optimized SLM**: Uses a Genetic Algorithm to search for the optimal phase rotation factors instead of exhaustive or random search.

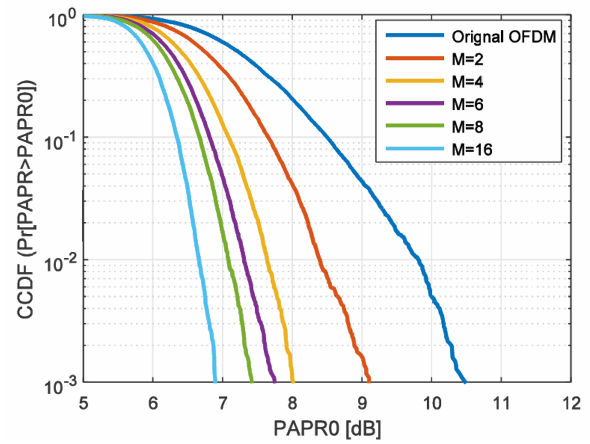

FIGURE 1.8: CCDF of PAPR for conventional SLM with different numbers of phase sequences U ($N = 128$, QPSK). Increasing U improves PAPR but with diminishing returns: the gap between $U = 8$ and $U = 16$ is only about 0.5 dB.

1.7 The Three Core Limitations of Classical SLM

Despite its elegance, classical SLM suffers from three fundamental limitations that motivate the ML-enhanced approaches explored in subsequent chapters.

1.7.1 Limitation 1: Computational Complexity

Each OFDM symbol requires U full N-point IFFT operations—one per candidate. The complexity of a single N-point IFFT is $\mathcal{O}(N \log_2 N)$, so the total SLM complexity per symbol is:

1.6

$$\mathcal{C}_{\text{SLM}} = \mathcal{O}\big(U \cdot N \log_2 N\big).$$

Complexity Numbers

For a 5G NR-like system with $N = 2048$ subcarriers and $U = 16$ candidates, each OFDM symbol requires $16 \times 2048 \times \log_2(2048) = 16 \times 2048 \times 11 = 360{,}448$ complex multiply-accumulate operations just for the PAPR reduction step. At a symbol rate of 14 symbols per slot $\times$ 2000 slots/s $=$ $28{,}000$ symbols/s, this amounts to over 10 billion operations per second—a non-trivial computational burden for real-time implementation.

1.7.2 Limitation 2: Side Information Overhead

The $\lceil \log_2 U \rceil$ SI bits per symbol carry a dual cost:

1. **Bandwidth:** They consume subcarriers (or dedicated signaling) that could otherwise carry user data, reducing spectral efficiency.
2. **Reliability:** An error in the SI bits causes *all* data bits in the symbol to be decoded incorrectly. Therefore, SI bits typically need stronger error protection than regular data, adding further overhead.

1.7.3 Limitation 3: Blind Detection Difficulty

To avoid the SI overhead entirely, one could attempt **blind detection**: the receiver tries all U phase sequences and determines the correct one without explicit SI. However:

- The receiver must perform U full FFT operations per symbol.
- Decision metrics (e.g., based on pilot symbols or higher-order statistics) are unreliable at low SNR.
- The probability of choosing the wrong phase sequence is non-negligible, especially for large U.

Blind SLM detection is therefore an *open problem* particularly well-suited to ML-based solutions, which can learn complex statistical patterns that classical detectors cannot capture.

1.8 Transition: From Classical Limitations to PRNet

The three core limitations of classical SLM identified in this chapter map directly onto the design objectives of PRNet [1], the deep-learning system studied in the following chapter:

1. **Complexity reduction** $\longrightarrow$ PRNet replaces the U-IFFT exhaustive search with a single neural network forward pass, making PAPR reduction a fixed-cost inference step independent of U [1].
2. **Side information elimination** $\longrightarrow$ PRNet trains encoder and decoder jointly, so the decoder implicitly encodes the full knowledge of the encoder mapping. No SI bits are ever transmitted [1].
3. **Intelligent constellation shaping** $\longrightarrow$ Rather than rotating a fixed phase alphabet, the PRNet encoder learns a data-dependent constellation geometry via gradient descent that directly minimises time-domain PAPR [1].

Key Insight

PRNet addresses all three SLM limitations simultaneously: it eliminates the multi-IFFT search, removes the side information requirement, and replaces random phase selection with an end-to-end learned constellation mapping. Chapter 2 develops the full PRNet system—architecture, loss functions, training procedure, and simulation results—in complete detail.

CHAPTER 2

PRNet: PAPR Reduction via Deep Autoencoder

From Searching to Learning

Classical SLM carries two practical costs: (i) computational overhead — U full IFFT operations and U PAPR comparisons for every single OFDM symbol, and (ii) side information — $\lceil \log_2 U \rceil$ bits that must reach the receiver without error, or the entire symbol is lost.

What if, instead of *searching* over a fixed set of phase sequences, we could *learn* a mapping that inherently produces low-PAPR signals? A mapping so deeply integrated into the transmitter that it needs no search, no candidate comparison, and no side information at all?

This is exactly what Kim et al. (2017) proposed with the **PAPR Reduction Network (PRNet)** — the first work to apply deep learning to the PAPR reduction problem in OFDM [1]. PRNet treats the entire transmitter–channel–receiver chain as a single **deep autoencoder**: the encoder learns a data-dependent constellation mapping that jointly minimises reconstruction error (BER) and PAPR, while the decoder learns the corresponding inverse mapping, completely eliminating the need for side information.

This chapter does three things: (1) explains the autoencoder paradigm and why it maps naturally onto OFDM, (2) derives the system model, loss functions, and two-stage training procedure from first principles, and (3) walks step by step through training iterations and deployment to build deep intuition for how the system actually learns and operates.

Note on System Parameters

To make the discussion concrete, all numerical values used in examples and derivations throughout this chapter (such as $N = 64$ subcarriers and 4-QAM/QPSK modulation yielding 128 raw information bits per OFDM symbol) are the simulation parameters explicitly adopted by the authors of the original PRNet paper [1]. In the paper's neural network formulation, those 64 QPSK symbol choices are represented as a 256-dimensional one-hot vector.

2.1 The Autoencoder Paradigm for OFDM

Before we look at PRNet's specific architecture, we need to understand *why* an autoencoder is such a natural fit for the PAPR problem. An **autoencoder** is a specialised neural network architecture designed to reconstruct its input at the output by forcing the data through a constrained "bottleneck" in the middle. Now let's see how this maps onto communication.

2.1.1 The Communication Chain as an Autoencoder

In any communication system, the transmitter transforms raw data into a signal suitable for the channel, and the receiver undoes that transformation to recover the original data. If we step back and look at the overall structure, this is *exactly* what an autoencoder does [1]:

- **The Encoder (transmitter):** takes the raw input data and transforms it into a new representation — the transmitted signal.
- **The Bottleneck (physical channel):** the signal must pass through the wireless channel, which adds noise, fading, and distortion. This is the constrained middle layer that the system cannot control.
- **The Decoder (receiver):** takes the corrupted version that survived the channel and attempts to reconstruct the original data.

The key insight of PRNet is that if we design both the transmitter and receiver as *jointly trained neural networks*, and place the real physical channel (modelled mathematically) between them, then we can train the pair end-to-end to minimise any objective we choose — including PAPR. The neural network at the transmitter will automatically discover a constellation shaping strategy that reduces PAPR, and the neural network at the receiver will automatically learn how to undo that shaping [1].

2.1.2 What PRNet Replaces in the OFDM Chain

It is important to be very precise about what PRNet does and does not replace. The architecture diagram below illustrates the complete PRNet system model.

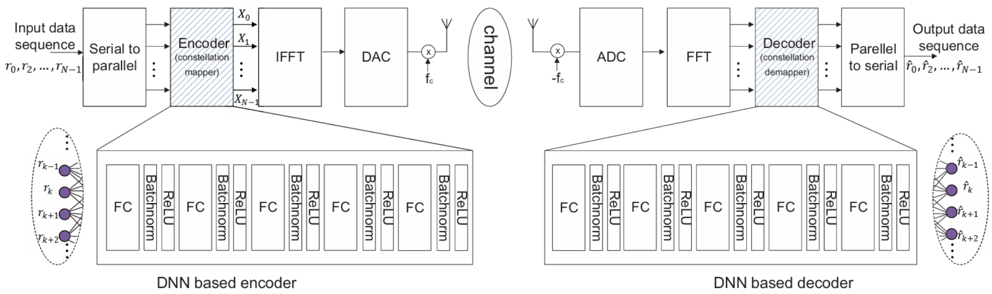

FIGURE 2.1: Structure of the proposed PRNet with DNN encoder and DNN decoder. Both are stacks of fully connected (FC) layers with Batch Normalization and ReLU activation. The encoder shapes the constellation before the IFFT to reduce PAPR; the decoder reconstructs the transmitted symbols after the channel and FFT. Reproduced from [1].

PRNet inserts two trainable Deep Neural Networks (DNNs) around the standard IFFT/FFT pair [1]:

- **Encoder (transmitter-side DNN):** replaces the standard constellation mapper. Instead of mapping bits to fixed QPSK or QAM points, it takes the input data and produces a new set of complex subcarrier values specifically shaped to yield low peaks after the IFFT.
- **Decoder (receiver-side DNN):** replaces the standard constellation de-mapper. It takes the received (noisy) frequency-domain subcarriers and directly reconstructs the original data bits.

Common Misconception

“PRNet replaces the IFFT with a neural network.”

This is a very common first reaction, but it is incorrect. The IFFT and FFT are still explicitly present in the signal chain. They remain fixed, standard mathematical operations sitting inside the PRNet pipeline. What the encoder replaces is the *constellation mapper* — instead of mapping bits to fixed QPSK points, it maps them to learned complex values that happen to produce lower peaks when passed through the IFFT. The IFFT itself is untouched [1].

2.1.3 Why No Side Information Is Needed

In classical SLM, the receiver must know *which* phase sequence the transmitter selected — this requires $\lceil \log_2 U \rceil$ bits of side information (SI) per OFDM symbol. If even one of those SI bits arrives in error, the receiver applies the wrong phase reversal and the entire OFDM symbol is destroyed.

PRNet eliminates this vulnerability entirely. Because the encoder and decoder are trained *jointly* over millions of iterations, the decoder's neural network weights implicitly encode the complete knowledge of what transformation the encoder applies. The decoder does not need to be told which mapping was used — it already knows, because it spent the entire training phase learning to invert exactly that mapping [1].

No Side Information Needed

In SLM, the decoding rule is external: "look up which phase sequence was used and reverse it." In PRNet, the decoding rule is *internal*: it is baked into the millions of learned weights inside the decoder network. No explicit communication between transmitter and receiver about the mapping is needed. This completely removes the SI overhead and the catastrophic failure mode of SI errors [1].

2.1.4 Why the Encoder Sees All Bits Simultaneously

In standard OFDM with QPSK, the 128 raw bits are chopped into 64 isolated pairs. Each pair is independently mapped to one of four constellation points, and that point is assigned to a single subcarrier. Bit pair #1 knows nothing about bit pair #2, and subcarrier #1 knows nothing about what is happening on subcarrier #2.

PRNet fundamentally breaks this isolation. The encoder is a **fully connected (FC)** neural network, which means every single output neuron receives input from every single input neuron. In the paper's implementation, the encoder receives the 256-entry one-hot representation of the 64 QPSK symbols; equivalently, it sees the whole 128-bit OFDM symbol at once. Each of the 64 output complex numbers is therefore a function of the **entire** OFDM input block — not just the 2 bits that would have been assigned to that subcarrier in standard QPSK [1].

This holistic processing is what gives PRNet its power. The encoder can detect dangerous patterns across the full input (e.g., long runs of identical bits that tend to cause constructive interference in the IFFT) and compensate for them by adjusting the amplitudes and phases of *all* 64 subcarriers jointly.

Holistic Processing

Standard OFDM maps bits to subcarriers independently: 2 bits $\to$ 1 subcarrier. PRNet maps the full OFDM input block to all subcarriers simultaneously: 64 QPSK symbol categories, or 128 underlying raw bits, $\to$ 64 learned subcarrier values. This dense dependence is what gives the encoder a chance to avoid PAPR spikes that arise from unfortunate whole-symbol patterns [1].

2.2 System Model: End-to-End Signal Flow

Now let's formalise the complete PRNet signal chain with full mathematical notation. We will walk through every step from the raw input bits to the recovered output, defining every symbol as it appears.

2.2.1 Step 1 — Preparing the Network Input

We consider an OFDM system with $N = 64$ subcarriers and QPSK modulation (i.e., $M = 4$ constellation points per subcarrier, representing $\log_2 4 = 2$ bits each). The total number of input bits per OFDM symbol is therefore $N \times \log_2 M = 64 \times 2 = 128$ bits [1].

In the paper's notation, the integer index for each subcarrier is $r_k \in \{0, 1, 2, 3\}$ (one of the 4 QPSK symbols), where $k = 0, 1, \ldots, N-1$ is the subcarrier index. The full input vector is $\boldsymbol{r} = [r_0,\, r_1,\, \ldots,\, r_{N-1}]^T$. In the actual neural network implementation, each symbol index r_k is represented as a **one-hot vector** of length $M = 4$: the r_k-th entry is set to 1 and the rest are 0. These one-hot vectors are concatenated to form the network's input [1]:

$$\boldsymbol{r}_{\text{input}} \in \mathbb{R}^{NM} = \mathbb{R}^{256}.$$

Our Rayleigh reproduction code uses a slightly different but conceptually equivalent representation: the QPSK symbols are stored directly as stacked real and imaginary coordinates, so the neural-network input dimension is $2N = 128$ rather than $NM = 256$. This changes the input encoding, not the core PRNet idea: the encoder still maps one complete OFDM data block into 64 learned complex subcarrier values.

For example, if $N = 4$ and the symbols are $[0, 2, 1, 3]$, the one-hot representation would be:

$$\boldsymbol{r}_{\text{input}} = [\underbrace{1, 0, 0, 0}_{r_0=0},\ \underbrace{0, 0, 1, 0}_{r_1=2},\ \underbrace{0, 1, 0, 0}_{r_2=1},\ \underbrace{0, 0, 0, 1}_{r_3=3}]^T.$$

Why One-Hot Encoding?

One-hot encoding prevents the neural network from incorrectly assuming that symbol index 3 is "three times as large" as symbol index 1. In reality, these indices are just labels for different constellation points — there is no meaningful ordering or magnitude relationship between them. One-hot encoding treats each symbol as an independent category, which is exactly what it is [1]. For an interactive explanation, see the Google Machine Learning Crash Course on One-Hot Encoding.

2.2.2 Step 2 — The Encoder: Learned Constellation Shaping

The encoder is a deep neural network that takes the one-hot input vector $\boldsymbol{r}_{\text{input}} \in \mathbb{R}^{NM}$ and outputs a vector of $2N = 128$ real numbers. These 128 numbers represent the real and imaginary parts of 64 complex subcarrier values [1]:

$$f(\boldsymbol{r};\, \theta_f) \in \mathbb{R}^{2N},$$

where $\theta_f = \{\boldsymbol{W}_1^f, \boldsymbol{b}_1^f, \ldots, \boldsymbol{W}_{L_f}^f, \boldsymbol{b}_{L_f}^f\}$ denotes all the learnable weights and biases of the encoder.

To use these $2N$ real numbers in the OFDM chain, we must convert them back into N complex symbols. We define the **unstacking operator** $\mathcal{C}(\cdot) : \mathbb{R}^{2N} \to \mathbb{C}^N$, which pairs up consecutive elements as real and imaginary parts:

$$\boldsymbol{X}_{\text{enc}} = \mathcal{C}\big(f(\boldsymbol{r};\, \theta_f)\big) \in \mathbb{C}^N,$$

where, concretely, the k-th encoded complex symbol is:

$$X_{\text{enc},k} = f(\boldsymbol{r})_{2k} + j \cdot f(\boldsymbol{r})_{2k+1}, \qquad k = 0, 1, \ldots, N-1.$$

These encoded symbols $\boldsymbol{X}_{\text{enc}}$ are *not* standard QPSK points. They are data-dependent, continuous-valued complex numbers that the encoder specifically crafted to produce a low-PAPR waveform after the IFFT. If you were to plot them on a constellation diagram, they would not form the neat four-point QPSK grid — instead, they would scatter in a seemingly irregular pattern that has been geometrically optimised for the IFFT's physics [1].

What the Encoder's Output Looks Like

In standard QPSK, every subcarrier gets one of exactly four points: $\{(+0.707, +0.707), (+0.707, -0.707), (-0.707, +0.707), (-0.707, -0.707)\}$. In PRNet, the encoder might output something like $(0.42, -0.91)$ for subcarrier #1, $(-1.13, 0.07)$ for subcarrier #2, $(0.68, 0.55)$ for subcarrier #3, and so on. These are not from any standard constellation — they are custom values that the network learned will avoid destructive IFFT interactions for this specific input bit pattern.

2.2.3 Step 3 — OFDM Modulation: The Standard IFFT

The encoded symbols are modulated onto the subcarriers via the standard IFFT [1]:

2.1

$$x[n] = \sum_{k=0}^{N-1} X_{\text{enc},k}\, e^{j2\pi nk/N}, \qquad 0 \le n \le N-1.$$

This is the standard IFFT operation — nothing has changed about OFDM modulation itself. The only difference is that the complex numbers $X_{\text{enc},k}$ entering the IFFT are no longer fixed QPSK points; they are the encoder's custom-shaped values.

Oversampling. PAPR depends on the *maximum* of the continuous-time waveform. If we only compute the IFFT at N samples, we can miss peaks that occur *between* samples. For a stricter approximation of continuous-time PAPR, the usual approach is to zero-pad the frequency-domain vector from length N to length NL, then compute an NL-point IFFT. With $N = 64$ and $L = 4$, this produces $64 \times 4 = 256$ time-domain samples.

This detail needs careful wording for PRNet. The standard PAPR evaluation in our Rayleigh reproduction uses $L = 4$, because this is the scientifically safer way to estimate peaks. However, the original paper does not clearly document oversampling in the PRNet CCDF calculation, and the authors' released code effectively evaluates the PRNet waveform on the N IFFT samples. Oversampling is therefore recommended for standard PAPR reporting, but it is not the reason the paper's red PRNet CCDF curve appears near 3 dB; that discrepancy is explained in the simulation results section below.

2.2.4 Step 4 — The Wireless Channel

After OFDM modulation, the time-domain signal is transmitted over the wireless channel. The paper models this as [1]:

$$\boldsymbol{y} = \mathbb{H}(\boldsymbol{x}) + \boldsymbol{\epsilon},$$

where $\mathbb{H}$ denotes the channel effects (the paper states Rayleigh fading [1]) and $\boldsymbol{\epsilon}$ is additive white Gaussian noise (AWGN) with noise power determined by the signal-to-noise ratio (SNR). The exact fading implementation matters in reproductions: the paper gives the high-level Rayleigh model, while executable code must decide whether fading is flat or frequency-selective and whether equalisation is performed at the receiver.

The Channel as a Fixed Mathematical Layer

From the neural network's perspective, the channel is not a trainable component — it is a *fixed mathematical function* sitting in the middle of the computation graph. During training, the channel is simulated by generating random noise and (optionally) random fading coefficients, and adding them to the transmitted signal. This simulation allows the gradients from the loss function to flow backward through the channel during backpropagation, because the channel is simply a sequence of differentiable mathematical operations (addition, multiplication) applied to the signal [1].

2.2.5 Step 5 — OFDM Demodulation and Decoding

At the receiver, the received time-domain signal $\boldsymbol{y}$ passes through the standard FFT to recover the frequency-domain subcarriers:

$$\boldsymbol{Y} = \mathrm{FFT}(\boldsymbol{y}) \in \mathbb{C}^{N}.$$

These received subcarriers are noisy, distorted versions of the encoder's output $\boldsymbol{X}_{\mathrm{enc}}$. Instead of a standard QAM de-mapper (which would look for the nearest QPSK point and reverse the mapping), PRNet feeds them into the **decoder DNN**.

The decoder first converts the complex symbols $\boldsymbol{Y}$ into a real vector by stacking real and imaginary parts (the inverse of the unstacking we did in Step 2), then processes this real vector through its own deep network to produce the output [1]:

$$\hat{\boldsymbol{r}} = g\big(\mathcal{S}(\boldsymbol{Y});\, \theta_g\big) \in \mathbb{R}^{NM},$$

where θ_g denotes the decoder's learnable parameters and $\mathcal{S}(\cdot) : \mathbb{C}^N \to \mathbb{R}^{2N}$ is the **stacking operator** that interleaves the real and imaginary parts into a single real vector.

The decoder's output $\hat{\boldsymbol{r}}$ has the same dimension as the original input ($\mathbb{R}^{NM}$ in the paper's one-hot formulation). Each group of $M = 4$ output values corresponds to one subcarrier, and the network outputs a real-valued **score** for each possible symbol index. These scores are not softmax probabilities; the final hard decision is made by picking the index with the largest score (i.e., $\hat{r}_k =$ arg max over the 4 values for subcarrier k).

How the Decoder Recovers the Data

For subcarrier #7, the decoder's output might be $[0.02,\ 0.01,\ 0.95,\ 0.02]$. The largest score is the third entry, so the receiver decides that the original symbol index was $r_7 = 2$. Since symbol index 2 in QPSK corresponds to the bit pair "10", the receiver records "10" for subcarrier #7. The decoder makes this decision purely from the 64 noisy complex numbers it received — it was never told what mapping the encoder used.

2.2.6 The Complete Chain in One Equation

The paper expresses the entire end-to-end PRNet chain in one compact equation [1]:

2.2

$$\hat{\boldsymbol{r}} = g \circ \text{FFT} \circ \mathbb{H} \circ \text{IFFT} \circ f(\boldsymbol{r})$$

Reading this from right to left: the input $\boldsymbol{r}$ enters the encoder f, passes through the IFFT, travels through the channel $\mathbb{H}$, is demodulated by the FFT, and finally enters the decoder g to produce the reconstructed output $\hat{\boldsymbol{r}}$.

The goal of training is to make $\hat{\boldsymbol{r}}$ as close to $\boldsymbol{r}$ as possible (low BER), while simultaneously ensuring that the IFFT output $x[n]$ has low PAPR.

In short, PRNet wraps the standard OFDM modulation/demodulation chain with two DNNs. The encoder reshapes the constellation *before* the IFFT to suppress peaks, and the decoder learns to invert that reshaping *after* the channel and FFT. The IFFT, FFT, and channel are all fixed — they are not replaced and their equations are unchanged from standard OFDM.

TABLE 2.1: Summary of PRNet signal dimensions for the paper-style one-hot formulation. With oversampling, $x[n]$ and $\boldsymbol{y}$ have length NL; with $L = 4$ this is 256 samples.

Symbol	**Domain / Shape**	**Meaning**
$\boldsymbol{r}$	$\mathbb{R}^{NM}$ (256)	One-hot encoded input (network input & target)
$f(\boldsymbol{r})$	$\mathbb{R}^{2N}$ (128)	Encoder output (stacked I/Q components)
$\boldsymbol{X}_{\text{enc}}$	$\mathbb{C}^{N}$ (64)	Encoded complex subcarrier symbols
$x[n]$	$\mathbb{C}^{NL}$ $(64L)$	IFFT output (time-domain waveform)
$\boldsymbol{y}$	$\mathbb{C}^{NL}$ $(64L)$	Received time-domain signal (after channel)
$\boldsymbol{Y}$	$\mathbb{C}^{N}$ (64)	FFT output (received subcarrier values)
$\hat{\boldsymbol{r}}$	$\mathbb{R}^{NM}$ (256)	Decoder output (reconstructed one-hot scores)

In the coordinate-based Rayleigh code used for our reproduction, the first and last rows are $\mathbb{R}^{2N}$ rather than $\mathbb{R}^{NM}$, because each QPSK symbol is represented by its real and imaginary coordinates instead of a one-hot category.

2.3 Network Architecture: FC + BatchNorm + ReLU

Now that we understand what the encoder and decoder are supposed to do, let's look at how they are built internally. Both networks have *identical* architectures composed of repeated sub-blocks, each consisting of three operations applied in sequence [1].

2.3.1 The Sub-Block: One Layer of Processing

Each sub-block transforms an input vector $\boldsymbol{h}^{(\ell-1)}$ into an output vector $\boldsymbol{h}^{(\ell)}$ through three stages:

Stage 1 — Fully Connected (FC) Layer. A fully connected layer (also called a dense layer) is a simple linear transformation where every input neuron is connected to every output neuron: it multiplies the input by a weight matrix and adds a bias vector. For a layer with d_{in} inputs and d_{out} outputs, the computation is [1]:

$$\boldsymbol{y}_{\text{FC}} = \boldsymbol{W}_m \boldsymbol{h}^{(\ell-1)} + \boldsymbol{b}_m,$$

where $\boldsymbol{W}_m \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is the weight matrix and $\boldsymbol{b}_m \in \mathbb{R}^{d_{\text{out}}}$ is the bias vector for the m-th sub-block. The hidden FC layers in PRNet use 2048 nodes — meaning $d_{\text{out}} = 2048$ for those layers, so a hidden weight matrix has $2048 \times d_{\text{in}}$ entries. These are the primary learnable parameters of the network [1].

Stage 2 — Batch Normalization (BN). As data flows through many layers in a deep neural network, the distribution of each layer's input can shift dramatically during training (a phenomenon called **internal covariate shift**). Batch Normalization fixes this by normalising the FC output to have zero mean and unit variance within each training mini-batch, then applying a learnable scale and shift [1]:

$$|\boldsymbol{y}_{\mathrm{FC}}|_{\mathrm{norm}} = \gamma \cdot \frac{\boldsymbol{y}_{\mathrm{FC}} - \mathbb{E}[\boldsymbol{y}_{\mathrm{FC}}]}{\sqrt{\mathrm{Var}[\boldsymbol{y}_{\mathrm{FC}}] + \nu}} + \beta,$$

where γ is a learnable scaling parameter, β is a learnable shifting parameter, and $\nu = 0.001$ is a small constant to prevent division by zero [1].

The practical effect is significant: without BatchNorm, training a 5-layer deep network is extremely slow because each layer's output statistics keep changing, forcing subsequent layers to constantly readapt. BatchNorm stabilises these statistics, allowing the use of higher learning rates and dramatically speeding up convergence.

Stage 3 — ReLU Activation. After normalisation, the values pass through the **Rectified Linear Unit** (ReLU) activation function [1]:

$$\phi(x) = \max(x, 0).$$

ReLU provides the *nonlinearity* that is absolutely essential for PRNet to work.

Why Nonlinearity Is Essential

Without nonlinear activation functions, no matter how many FC layers we stack, the entire network would collapse into a single matrix multiplication: $\boldsymbol{W}_5\boldsymbol{W}_4\boldsymbol{W}_3\boldsymbol{W}_2\boldsymbol{W}_1\boldsymbol{r} + \boldsymbol{b} = \boldsymbol{W}_{\mathrm{equiv}}\boldsymbol{r} + \boldsymbol{b}$. A single linear transformation can only rotate and scale the constellation — it cannot perform the complex, nonlinear warping required to simultaneously avoid all PAPR-inducing patterns. The paper explicitly notes that the constellation shaping problem is a "multivariate non-convex problem that is NP-hard" [1], which means no linear method can solve it. ReLU is what gives the network the expressive power to learn the nonlinear mapping.

2.3.2 The Complete Sub-Block

Putting all three stages together, the transformation performed by the ℓ-th sub-block is [1]:

$$\boldsymbol{h}^{(\ell)} = \phi\Big(\mathrm{BN}\big(\boldsymbol{W}^{(\ell)}\boldsymbol{h}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}\big)\Big), \qquad \phi(\cdot) = \mathrm{ReLU}(\cdot).$$

2.3.3 Full Encoder and Decoder

The paper describes the encoder as $L_f = 5$ FC–BN–ReLU sub-blocks chained together. Its mathematical expression is [1]:

2.3

$$f(\boldsymbol{r}) = \phi_{L_f}\Big(|\boldsymbol{W}^f_{L_f}\,\phi_{L_f-1}(\cdots\phi_1(|\boldsymbol{W}^f_1\,\boldsymbol{r} + \boldsymbol{b}^f_1|_{\text{norm}})\cdots) + \boldsymbol{b}^f_{L_f}|_{\text{norm}}\Big),$$

where $\boldsymbol{W}^f_{l_f}$ and $\boldsymbol{b}^f_{l_f}$ are the weights and biases of the l_f-th FC in the encoder, and $|\cdot|_{\text{norm}}$ denotes the batch normalisation operation. The released code uses the same design idea but implements the physical I/Q output as a final linear projection, so the final layer has dimension $2N$ rather than 2048 hidden nodes.

The decoder has the exact same structure — $L_g = 5$ sub-blocks — but with its own separate set of weights and biases ($\boldsymbol{W}^g_{l_g}$, $\boldsymbol{b}^g_{l_g}$). The encoder and decoder are symmetric in topology but **not** in learned weights — each network independently discovers its own optimal parameters during training [1].

The **final output layer** of each network deserves special mention: it is a plain FC layer with **no activation function** (i.e., linear output). This is because the output must be unconstrained real values — the encoder needs to produce arbitrary I/Q amplitudes, and the decoder needs to produce arbitrary scores for each possible symbol. If we applied ReLU to the output, all negative values would be clamped to zero, which would prevent the encoder from producing negative I/Q components.

In short, each encoder and decoder is a deep FC network with 2048-node hidden processing blocks, BatchNorm, ReLU nonlinearities, and a linear output projection for the final transmitted I/Q values or reconstructed symbol scores. The nonlinear hidden stack gives the network enough expressive power to learn the complex, NP-hard constellation shaping that reduces PAPR.

2.4 The Loss Function and Training Procedure

Now we arrive at the core of PRNet: *how* the encoder and decoder actually learn. The key challenge is that we must optimise for two competing objectives simultaneously — and the tension between them is what makes PRNet's training procedure so carefully designed.

2.4.1 Two Competing Objectives

Let's state the two goals plainly before introducing any math:

1. **Objective 1 — Recoverability (low BER):** The decoder must reliably reconstruct the original data from the noisy received signal. If we only cared about this, we would train a standard autoencoder — it would learn to survive channel noise, but would do nothing about PAPR.
2. **Objective 2 — Peak suppression (low PAPR):** The encoder's output must produce a time-domain waveform with low peaks after the IFFT. If we only cared about this, we could flatten the signal entirely to a constant envelope — but then all subcarriers would carry the same information, and the decoder would have no way to distinguish different bit patterns.

These two objectives are fundamentally in tension: aggressively reducing PAPR requires distorting the constellation geometry, which makes the decoder's job harder and increases BER. PRNet manages this tension through a carefully designed **joint loss function** that balances both goals [1].

2.4.2 Loss 1 — Reconstruction Loss (MSE)

The first loss function measures how well the decoder recovers the original input. It computes the **Mean Squared Error (MSE)** between the original input vector $\boldsymbol{r}$ and the decoder's output $\hat{\boldsymbol{r}}$ [1]:

2.4

$$\mathcal{L}_1(\boldsymbol{r}, \hat{\boldsymbol{r}}) = \frac{1}{NM}\|\boldsymbol{r} - \hat{\boldsymbol{r}}\|_2^2 = \frac{1}{NM}\sum_{i=0}^{NM-1}\left(r_i - \hat{r}_i\right)^2.$$

This is the standard autoencoder reconstruction loss. It computes the squared difference between each element of the original one-hot input and the decoder's corresponding output, then averages over all NM elements.

When this loss is large, it means the decoder is producing outputs that are far from the original data — in other words, many bits are being decoded incorrectly. When this loss is small, the decoder is accurately recovering the transmitted data, which translates directly to low BER.

Why MSE and Not Cross-Entropy?

In classification problems (e.g., "is this image a cat or a dog?"), cross-entropy is the standard loss function. The paper instead trains PRNet as an autoencoder that reconstructs its input vector directly, so it uses MSE between the target one-hot vector and the decoder's output scores [1]. A classification-style formulation with a softmax decoder would be possible in principle, but it is not the loss used in the original PRNet work or in our reproduction.

2.4.3 Loss 2 — PAPR Penalty

The second loss function directly measures the PAPR of the time-domain waveform. PAPR is the ratio of peak instantaneous power to average power. Written with an explicit oversampling factor L, the standard power-PAPR loss is:

2.5

$$\mathcal{L}_2(\boldsymbol{r}) = \mathrm{PAPR}\{x[n]\} = \frac{\max\limits_{0\leq n\leq NL-1} |x[n]|^2}{\frac{1}{NL}\sum\limits_{n=0}^{NL-1} |x[n]|^2}.$$

For scientifically strict reporting, both the maximum and the mean should be computed over the oversampled waveform. In the original paper, Eq. (6) writes the PAPR term compactly as $\mathrm{PAPR}\{\mathrm{IFFT}(f(\boldsymbol{r}))\}$ without stating the oversampling detail. Our Rayleigh reproduction trains the PAPR term with $L = 4$ and uses the physical power ratio above. This loss term is what actively pushes the encoder to shape the constellation such that the IFFT output has smaller peaks.

How Gradients Flow Through PAPR

At first glance, computing a gradient through the max function might seem impossible. But in practice, max is differentiable almost everywhere — its gradient is simply 1 at the position of the maximum and 0 everywhere else. The gradient tells the encoder: "the peak power occurred at time sample n^*; the subcarrier values that contributed most to sample n^* should be adjusted." Through the chain rule, this gradient propagates backward through the IFFT (which is just a linear operation) all the way to the encoder's weights [1].

2.4.4 The Joint Loss Function

The final PRNet loss is a weighted sum of the two individual losses (paper Eq. (7)) [1]:

2.6

$$\mathcal{L}(\boldsymbol{r}, \hat{\boldsymbol{r}}) = \mathcal{L}_1(\boldsymbol{r}, \hat{\boldsymbol{r}}) + \lambda\, \mathcal{L}_2(\boldsymbol{r}),$$

where $\lambda > 0$ is a scalar weight that controls the trade-off between reconstruction accuracy and PAPR reduction.

Let's understand exactly what λ does:

- **If $\lambda = 0$:** The PAPR penalty is completely ignored. The network is trained as a pure autoencoder. It will reconstruct symbols perfectly but will not reduce PAPR at all.
- **If λ is very large (e.g., $\lambda = 1$):** The PAPR penalty dominates. The encoder is pushed extremely hard to flatten the time-domain waveform, but this aggressive constellation distortion makes it nearly impossible for the decoder to distinguish between different bit patterns. BER skyrockets.
- **If $\lambda = 0.01$ (the paper's chosen value):** The PAPR term is multiplied by 10^{-2} before being added to the reconstruction term. The actual balance depends on the numerical scales of $\mathcal{L}_1$ and $\mathcal{L}_2$, but empirically this value lets the encoder focus primarily on decodability while still nudging the constellation toward lower peaks [1].

The λ Knob

λ is the single most important hyperparameter in PRNet. It controls a fundamental, inescapable trade-off: *more PAPR reduction necessarily costs some BER performance, and vice versa.* This trade-off is not a flaw of PRNet — it is a physical reality of the OFDM system. Any method that claims to reduce PAPR without any BER cost is either adding redundancy (like coding) or is operating in a regime where the PAPR reduction is minimal [1].

2.4.5 Two-Stage Training Procedure

PRNet is trained as a **denoising autoencoder**: during training, noise is deliberately injected into the signal (simulating the channel) so that the decoder learns a robust inverse mapping that can handle real channel conditions. The amount of training noise is controlled by the **corruption level** η [1]:

$$\eta = \frac{\mathbb{E}\left[|\epsilon|^2\right]}{\mathbb{E}\left[|\boldsymbol{r}|^2\right]},$$

where ϵ is the injected noise power and $\boldsymbol{r}$ is the signal power. This is essentially a noise-to-signal ratio, expressed in dB. For example, $\eta = -15$ dB means the noise power is $10^{-1.5} \approx 0.032$ times the signal power — a relatively mild corruption.

Choosing η correctly is critical, because it determines the size of the "noise region" that the decoder must learn to handle around each constellation point [1]:

- **Too little noise (η very negative, e.g., -30 dB):** The decoder only learns to recover symbols from a tiny noise region very close to each constellation point. At test time, when real channel noise is larger than what the network trained on, the decoder encounters unfamiliar distortion and fails.
- **Too much noise (η too high, e.g., 0 dB):** The noise regions of adjacent constellation points overlap heavily during training. The decoder becomes confused and cannot learn to reliably distinguish between neighbouring points.
- **The sweet spot ($\eta = -15$ dB):** The noise is strong enough that the decoder learns robust decision boundaries, but weak enough that the constellation points remain distinguishable during training.

To find this sweet spot, Kim et al. use a **two-stage training procedure** [1]:

1. **Stage 1 — Choose $\eta^\star$ (reconstruction only):** Train multiple separate PRNet models using *only* the reconstruction loss $\mathcal{L}_1$ (i.e., $\lambda = 0$, no PAPR penalty) for a range of η values (e.g., $\eta \in \{0, -5, -10, -15, -20, -25, -30\}$ dB). After training each model, evaluate its BER across a full SNR sweep. Select the η that yields the lowest overall BER. The paper finds $\eta^\star = -15$ dB [1].
2. **Stage 2 — Joint training at $\eta^\star$:** Fix $\eta = \eta^\star = -15$ dB and train the final PRNet with the full joint loss $\mathcal{L} = \mathcal{L}_1 + \lambda\,\mathcal{L}_2$. This is the final model that simultaneously reduces PAPR and maintains low BER [1].

Algorithm 1: Two-stage PRNet training procedure (Kim et al., 2017 [1]).

Input : Candidate corruption levels $\{\eta\}$, PAPR weight λ, steps T, batch size B

Output : Trained parameters (θ_f, θ_g)

`// Stage 1: choose` $\eta^\star$ `using` $\lambda = 0$

foreach η *in* $\{\eta\}$ **do**
- Initialise PRNet parameters (θ_f, θ_g)
- **for** $t = 1$ **to** T **do**
  - Sample a batch of B random QPSK symbol indices $\rightarrow \boldsymbol{r}$
  - Forward pass: $\hat{\boldsymbol{r}} \leftarrow \text{PRNet}(\boldsymbol{r}; \theta_f, \theta_g, \eta)$
  - Compute $\mathcal{L}_1(\boldsymbol{r}, \hat{\boldsymbol{r}})$ and update (θ_f, θ_g) via Adam
- **end**
- Evaluate BER over an SNR sweep and record result for this η

end

Select $\eta^\star$ that minimises BER

`// Stage 2: joint training at` $\eta^\star$

Initialise or continue training the final PRNet parameters (θ_f, θ_g)

for $t = 1$ **to** T **do**
- Sample a batch of B random QPSK symbol indices $\rightarrow \boldsymbol{r}$
- Forward pass: obtain $\hat{\boldsymbol{r}}$ and time-domain $x[n]$
- Compute $\mathcal{L} = \mathcal{L}_1(\boldsymbol{r}, \hat{\boldsymbol{r}}) + \lambda\, \mathcal{L}_2(x[n])$
- Update (θ_f, θ_g) via Adam on $\nabla \mathcal{L}$

end

The reason for separating the two stages is pedagogical and practical. Stage 1 ensures the decoder develops solid noise-handling foundations *before* the encoder starts actively distorting the constellation for PAPR reduction. If we started with the PAPR penalty active from the very beginning, the encoder would immediately start warping the constellation, but the decoder — which has random, untrained weights — would have no ability to follow those distortions. The result would be chaotic, with neither objective being optimised properly.

In short, Stage 1 builds robust reconstruction capability; Stage 2 then carefully introduces the PAPR objective on top of that solid foundation.

2.4.6 Backpropagation Through the Channel

A natural question arises: "How can we backpropagate gradients through the wireless channel? The channel is a physical process, not a neural network layer."

Common Misconception

"You cannot backpropagate through a channel."

For an AWGN channel, the received signal is $\boldsymbol{y} = \boldsymbol{x} + \boldsymbol{\epsilon}$, where $\boldsymbol{\epsilon}$ is random noise that does *not* depend on the encoder's parameters. The gradient of the loss with respect to $\boldsymbol{x}$ is simply $\partial\mathcal{L}/\partial\boldsymbol{x} = \partial\mathcal{L}/\partial\boldsymbol{y}$, because $\partial(\boldsymbol{x}+\boldsymbol{\epsilon})/\partial\boldsymbol{x} = 1$ — the additive noise passes through as a constant during differentiation [1].

Even for a multiplicative fading channel ($\boldsymbol{y} = h\boldsymbol{x} + \boldsymbol{\epsilon}$), the backpropagated gradient is scaled by the channel coefficient (more precisely, by the adjoint coefficient h^* in complex notation). This is perfectly computable as long as the fading coefficient h is sampled independently of the transmitted signal and treated as fixed during that forward/backward pass. The channel acts as a known differentiable layer [1].

The same applies to the IFFT and FFT: they are linear operations (matrix multiplications by the DFT matrix), and the gradient of a linear operation is simply its transpose. So gradients flow cleanly from the loss function backward through the decoder, through the FFT, through the channel, through the IFFT, and all the way back to the encoder's first weight matrix. Every single weight in the entire system gets a gradient signal telling it how to adjust.

2.4.7 Stepping Through the First Training Iterations

To truly understand *how* PRNet learns, it helps to walk through the very first training iterations in detail. This reveals how the system goes from complete chaos to a highly optimised mapping.

Iteration 1: The Chaos Phase

Before any data is processed, the system initialises. Every single weight and bias inside both the encoder and decoder is set to a small, completely random number (using Xavier initialisation [1]). At this point, the network is essentially performing meaningless arithmetic.

1. **Generate input:** We generate a batch of 400 random QPSK symbol sequences. Each sequence consists of 64 random symbols (128 random bits), one-hot encoded into a 256-dimensional vector $\boldsymbol{r}$.
2. **Encoder (random math):** The 256-element input vector enters the encoder. Because the weights are random, the five FC $\rightarrow$ BN $\rightarrow$ ReLU layers perform meaningless matrix multiplications. The encoder outputs 128 real numbers

that bear no meaningful relationship to the input bits — they are essentially random I/Q values.

3. **IFFT:** These 64 random complex numbers enter the standard IFFT. Because the numbers are random, some of them will likely align constructively, producing a time-domain waveform with a massive power spike. The PAPR is very high.
4. **Channel:** We simulate the channel by mathematically adding complex Gaussian noise at the level dictated by $\eta = -15$ dB. The signal is now both high-PAPR *and* noisy.
5. **FFT:** The noisy time-domain signal is converted back into 64 complex numbers via the FFT. These are noisy, corrupted versions of the encoder's already-random output.
6. **Decoder (random guessing):** The decoder receives these 64 corrupted complex numbers (stacked into a 128-element real vector) and passes them through its own random weights. Its 256-element output is essentially random: each group of 4 values for each subcarrier is an unstructured set of scores, so the argmax decision is roughly a uniform random guess among the four QPSK symbols.
7. **Loss calculation:** The system evaluates its performance:
   - **Reconstruction loss $\mathcal{L}_1$:** The MSE between the original one-hot input and the decoder's random output is enormous — the decoder is getting most bits wrong.
   - **PAPR loss $\mathcal{L}_2$:** The PAPR of the time-domain waveform is very high (often equivalent to 10+ dB when expressed in dB).
   - **Total loss:** $\mathcal{L} = \mathcal{L}_1 + 0.01 \times \mathcal{L}_2$ = a very large number.
8. **Backpropagation:** The system takes this total loss and works backward through the entire chain using the chain rule. For every single weight in both the encoder and decoder, it computes the gradient: "If I had shifted this specific weight by a tiny amount, would the total loss have been slightly lower or slightly higher?" It then updates each weight by a small step (controlled by the Adam optimiser's learning rate of 10^{-4}) in the direction that reduces the loss.

After this first iteration, the 400 input sequences are **discarded**. They were training examples, not real data. The only thing that persists is the tiny adjustment made to the weights.

Iteration 2: The First Micro-Improvement

A brand new batch of 400 random sequences is generated. The same process repeats, but now the weights have been nudged slightly:

- The encoder's output is infinitesimally less chaotic — perhaps one or two of the 64 complex numbers are now in slightly more favourable positions.
- The PAPR might be 0.01% lower than it would have been with the original random weights.
- The decoder might correctly identify 65 out of 128 bits instead of the expected 64 from pure chance.
- The total loss is still massive, but measurably lower than Iteration 1.

Backpropagation runs again, computing new gradients based on this specific batch, and nudges the weights another tiny step.

Iterations 3 through 500,000: Building the Mapping

This exact loop repeats 500,000 times. With each iteration:

- **The encoder's weights migrate** toward mathematical operations that consistently produce complex numbers avoiding constructive interference in the IFFT. Over tens of thousands of gradient updates, each weight settles into a value that contributes to "safe" constellation shapes.
- **The decoder's weights migrate** toward operations that can reliably decode the specific, non-standard constellation shapes that the encoder is settling on — even through channel noise. The encoder and decoder are co-evolving: as the encoder changes its mapping, the decoder adapts, and vice versa.

By the end of training, the weights have converged. The total loss is orders of magnitude smaller than at Iteration 1. The encoder now produces a consistent, deterministic mapping from any 128-bit input to 64 complex numbers that yield low PAPR, and the decoder can reliably reconstruct the original bits from the noisy received versions of those complex numbers.

The Scale of Training

With 500,000 iterations and a batch size of 400, the network processes $500{,}000 \times 400 = 200{,}000{,}000$ (200 million) random OFDM symbols during training. Each symbol consists of 128 bits, so the network sees $200{,}000{,}000 \times 128 = 25.6 \times 10^9$ total bits. Yet the input space has $2^{128} \approx 3.4 \times 10^{38}$ possible bit sequences — the network sees only a vanishingly small fraction. Despite this, it learns to handle **all** possible inputs through generalisation: it discovers the underlying geometric rules of PAPR avoidance, not a memorised lookup table.

In short, training is a process of gradual, iterative refinement. The system starts from complete randomness and, through millions of tiny gradient-guided corrections, sculpts its weights into a precise mathematical function that maps any input bit sequence to a low-PAPR constellation shape while remaining decodable through channel noise.

2.5 The Deployed System: Inference, Generalization, and Resilience

Once training is complete, the system transitions from the laboratory phase to the deployment phase. This transition changes the system's behaviour fundamentally.

2.5.1 Frozen Weights and Pure Forward Passes

After training, the weights of both the encoder and decoder are **permanently frozen**. There is no more loss computation, no more gradient calculation, and no more back-propagation. The network becomes a static, deterministic mathematical function that simply evaluates in one direction — the forward pass [1].

A single live transmission proceeds as follows:

1. A block of 128 raw data bits arrives at the transmitter.
2. The bits are mapped to 64 QPSK symbol choices and represented either as the paper's 256-dimensional one-hot vector r or as stacked I/Q coordinates in our reproduction code.
3. The frozen encoder performs a single forward pass — just five rounds of matrix multiplication, batch normalisation using learned inference statistics, and ReLU — and outputs 128 real values (64 complex symbols).
4. The 64 complex symbols enter the standard IFFT. Because the encoder has been trained to shape them appropriately, the resulting time-domain waveform has low PAPR.

5. The waveform is transmitted over the physical channel and corrupted by real noise and fading.
6. At the receiver, the FFT converts the received signal back into 64 noisy complex symbols.
7. The frozen decoder performs a single forward pass and outputs symbol scores. The receiver picks the highest score for each subcarrier group, or the nearest coordinate in the coordinate-based implementation, to recover the original 128 bits.

Determinism in Deployment

The frozen encoder is a *deterministic* function. If the exact same 128-bit sequence is fed in on Monday and again on Friday, the encoder will produce the *exact* same 64 complex numbers both times. There is no randomness, no search, and no trial and error. The system trusts the mapping it spent 500,000 training iterations perfecting and executes it in a single pass. This is what makes PRNet's inference so fast — it is just a sequence of matrix multiplications [1].

2.5.2 Generalization to Unseen Bit Sequences

A legitimate concern is: "The input space has $2^{128} \approx 3.4 \times 10^{38}$ possible bit sequences. The network only saw 200 million of them during training. What happens when it encounters a sequence it has never seen?"

The answer lies in the fundamental nature of neural networks. The encoder is not a lookup table that memorises a mapping for each training sequence. It is a massive **algebraic function** — a composition of matrix multiplications and ReLU operations — whose parameters were tuned to perform well *in general*, not on specific inputs.

During training, the network learned abstract geometric rules:

- *Certain patterns of 1s and 0s* (e.g., long runs of identical bits) tend to produce high PAPR when mapped naively.
- *Specific adjustments to subcarrier phases and amplitudes* counteract those dangerous patterns.
- *These adjustments must be large enough* to reduce PAPR but small enough that the decoder can still distinguish the constellation points through channel noise.

When an unseen 128-bit sequence arrives, the matrix multiplications execute on it mechanistically. The resulting 64 complex numbers follow the learned geometric rules — not because the network "remembers" this input, but because the weight values encode the general principles of PAPR avoidance. This ability to perform well on inputs not seen during training is called **generalization**, and it is the foundational property that makes neural networks useful [1].

2.5.3 Dense Mixing and Channel Robustness

The fully connected architecture of PRNet has an important algebraic property: each encoded subcarrier can depend on the entire input block. This dense mixing is one reason a jointly trained decoder can learn a non-standard constellation geometry rather than merely demapping independent QPSK points.

It is tempting to describe this as built-in frequency diversity, but that would be stronger than what the paper proves. PRNet does not add redundancy or explicitly implement a diversity code; it still transmits one OFDM symbol's worth of learned subcarrier values. A severe fade or modelling mismatch can therefore still damage the received representation. What the dense architecture *can* do is learn a mapping that is robust to the channel distribution seen during training, provided the decoder is trained under a matching channel model [1].

> **Dense Mixing Is Not a Diversity Guarantee**
>
> Every PRNet output depends on the whole input block, so the learned geometry is global rather than subcarrier-by-subcarrier. This may improve robustness under the simulated Rayleigh channel, but it should not be reported as a formal frequency-diversity scheme unless a separate analysis establishes diversity order, coding gain, and performance under frequency-selective fading.

In short, the deployed PRNet system is fast (single forward pass), deterministic (same input always produces same output), robust to unseen inputs through learned geometric rules, and channel-dependent: it performs best when the deployment channel matches the one used during training.

2.6 Simulation Parameters and Results

Having explained the architecture, loss functions, and training procedure in detail, we now examine the simulation results that demonstrate PRNet's performance. The plotted figures in this section are reproduced from Kim et al. [1]. Where our audit of the released code changes how a result should be interpreted, especially for the CCDF curve, that caveat is stated explicitly.

2.6.1 Simulation Parameters

The simulation parameters table below summarises the key values used in the PRNet simulations.

TABLE 2.2: PRNet simulation parameters [1].

Parameter	Value
Number of subcarriers (N)	64
Modulation	4-QAM/QPSK
Network input representation	$NM = 256$ one-hot entries
PAPR sampling	N in released code; $L = 4$ recommended for standard PAPR
Hidden processing blocks per DNN	5 in paper; final output is linear in code
Hidden nodes per layer	2048
Activation function	ReLU (hidden), Linear (output)
Batch normalisation	Yes, after every FC layer
Optimiser	Adam in released code
Learning rate	10^{-4} in released code
Batch size	400
Training steps	500,000
Optimal corruption level ($\eta^\star$)	-15 dB
PAPR weight (λ)	0.01
CCDF evaluation size	50,000 random OFDM input sequences
PTS comparison	4 sub-blocks, 4 phase factors

2.6.2 BER Performance (Paper Figure 2)

The BER vs. SNR curves below show the Bit Error Rate versus the Signal-to-Noise Ratio for PRNet and the conventional PAPR reduction schemes used for comparison.

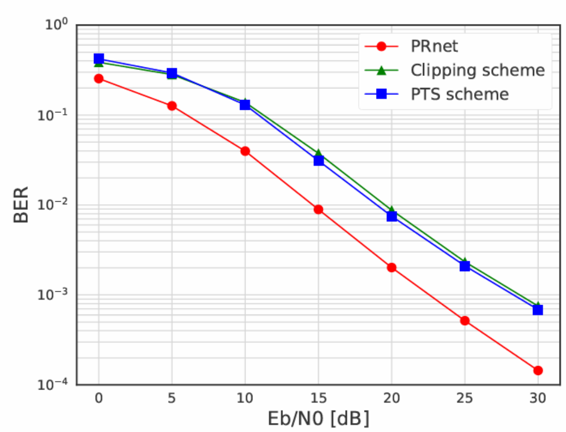

FIGURE 2.2: BER vs. SNR of conventional PAPR reduction schemes and PRNet. PRNet achieves the lowest BER among the plotted schemes because the decoder is trained jointly with the encoder to invert the learned constellation shaping. Reproduced from [1].

Several observations are worth highlighting:

- **Clipping** reduces PAPR but introduces in-band distortion, so its BER is degraded, especially at high SNR where the clipping noise becomes the dominant error source.
- **PTS** is distortionless in principle, but it requires side information and multiple IFFT computations.
- **PRNet** achieves BER that is *lower* than the plotted PAPR-reduction baselines. This is possible because the encoder-decoder pair is trained jointly: the encoder shapes the constellation in a way that the decoder can exploit, effectively learning an implicit coding gain under the simulated channel [1].

Why PRNet's BER Is Lower Than Conventional OFDM

This result often surprises first-time readers: how can a PAPR-reduction method avoid the BER penalty usually seen with clipping? The explanation is that the encoder does not merely apply phase rotations (like SLM) — it completely reshapes the constellation geometry. The jointly trained decoder is specifically designed to decode this custom geometry, giving it an advantage over a receiver that treats the shaping as distortion. In effect, the autoencoder has learned a form of *joint source-channel coding* that is optimised for the specific channel conditions it was trained on [1].

2.6.3 PAPR Reduction (Paper Figure 3)

The CCDF plot below shows the probability that the PAPR of a randomly generated OFDM symbol exceeds a given threshold.

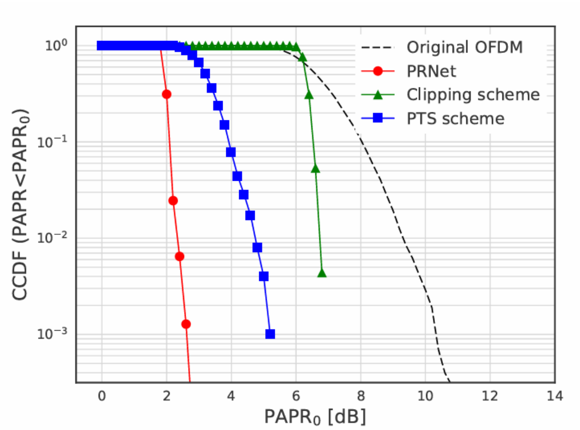

FIGURE 2.3: CCDF of PAPR as reported by the original PRNet paper. The PRNet curve shifts left, but the released code indicates that the red PRNet curve is based on a peak-amplitude proxy rather than the standard power-PAPR definition. Reproduced from [1].

Reading the CCDF at 10^{-3}

The most standard way to read PAPR performance from a CCDF plot is to pick a fixed probability level — commonly 10^{-3} (meaning "1 in 1000 symbols") — and read off the PAPR value on the x-axis. In the paper's plotted Fig. 3, conventional OFDM is around 10–11 dB at CCDF $= 10^{-3}$, while the red PRNet curve is near 3 dB. That visual result matches the paper's textual claim that $\mathrm{Prob}\{\mathrm{PAPR} > 3\ \mathrm{dB}\} < 0.1\%$ [1].

The PRNet curve is shifted *significantly* to the left compared to conventional OFDM. The paper reports that with PRNet, the probability of PAPR exceeding 3 dB is less than 0.1% [1]. In practical terms, this means the High-Power Amplifier at the transmitter can operate with substantially less Input Back-Off, directly improving power efficiency.

Common Misconception

"The paper's red CCDF curve is standard oversampled PAPR."

This should be treated with caution. The physical PAPR definition is

$$\mathrm{PAPR}_{\mathrm{dB}} = 10\log_{10}\frac{\max_n |x[n]|^2}{\frac{1}{N_s}\sum_n |x[n]|^2}.$$

In the authors' released TensorFlow code, however, the PRNet CCDF samples are generated from $\max_n |x[n]|$ and then plotted as $10\log_{10}(\cdot)$. For a normalised waveform, that peak-amplitude proxy is roughly one half of the usual power-PAPR value in dB. This explains why a scientifically standard evaluation of our Rayleigh-trained PRNet gives a PRNet value near 6 dB at CCDF $= 10^{-3}$, while the paper-compatible proxy appears near 3 dB. Therefore, the paper figure is reproducible, but the red curve should not be cited as standard power PAPR without this qualification.

2.6.4 Optimal Corruption Level (Paper Figure 4)

The BER-versus-corruption curves below show the BER of PRNet models trained at different corruption levels η, validating the two-stage training procedure.

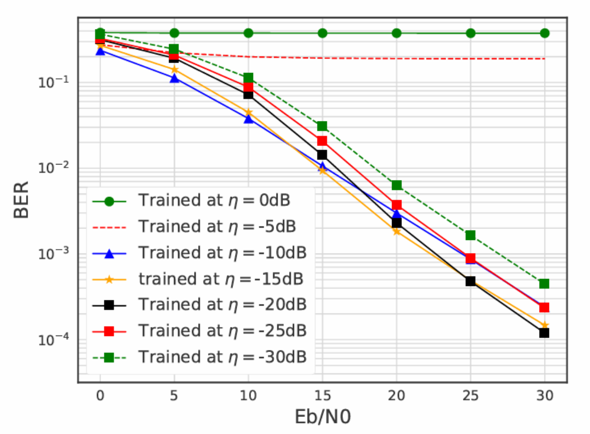

FIGURE 2.4: BER of PRNet trained at different corruption levels η. The curve for $\eta = -15$ dB consistently yields the lowest BER across the entire SNR range. Reproduced from [1].

The results confirm the training intuition developed above:

- $\eta = -30$ **dB (too little noise):** The decoder learned very tight decision boundaries during training and cannot handle the larger noise encountered at test time. BER is poor across all SNR values.
- $\eta = 0$ **dB (too much noise):** The noise during training was so severe that the decoder could not learn meaningful decision boundaries at all. BER is also poor.
- $\eta = -15$ **dB (the sweet spot):** The noise was strong enough to force the decoder to learn robust boundaries, but gentle enough that the constellation points remained distinguishable. This η yields the lowest BER across *all* tested SNR values, validating the paper's choice of $\eta^\star = -15$ dB [1].

2.6.5 The BER–PAPR Trade-off via λ (Paper Figure 5)

The trade-off curves below show how varying λ affects both BER and average PAPR at a fixed SNR of 20 dB.

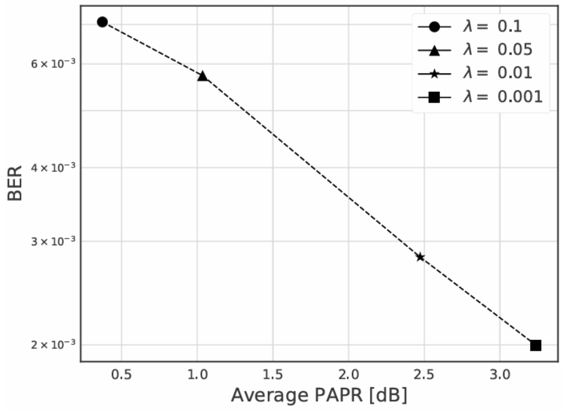

FIGURE 2.5: BER vs. average PAPR by varying λ when SNR $= 20$ dB. Larger λ reduces PAPR but increases BER, confirming the fundamental trade-off. Reproduced from [1].

The results visualise the trade-off that λ controls:

- As λ increases from 0 to 0.1, the average PAPR decreases monotonically — the encoder sacrifices more constellation clarity for peak suppression.
- Simultaneously, BER increases — the decoder's reconstruction task becomes harder because the constellation is increasingly distorted.
- At $\lambda = 0.01$, the paper reports a practical sweet spot: meaningful PAPR reduction while keeping BER competitive with the conventional PAPR-reduction baselines [1].

2.6.6 Computational Overhead

At inference time, PRNet replaces the U-fold *search* of SLM/PTS with a single forward pass through two DNNs plus one IFFT/FFT pair. The computational complexity of each forward pass through the encoder (or decoder) is [1]:

$$\mathcal{C}_{\text{DNN}} = \mathcal{O}(L \cdot K \cdot M),$$

where L is the number of hidden layers, K is the number of hidden nodes per layer, and M is the input dimension. For PRNet with $L = 5$ and $K = 2048$, this is a fixed cost — it does not grow with the number of candidates U like SLM does.

Kim et al. report measured execution times [1]:

TABLE 2.3: Comparison of execution times for different PAPR reduction methods [1].

Method	Execution time (μs)
PTS	2,890
Clipping and filtering	1,415
PRNet (without parallelism)	2,304
PRNet (with GPU)	480

With GPU-based parallel computation, PRNet is approximately **6 times faster** than PTS and **3 times faster** than clipping and filtering, while simultaneously achieving better BER and PAPR performance. Even without parallelism, PRNet is comparable in speed to PTS while being fundamentally simpler — it has no search loop, no candidate comparison, and no side information.

In short, the simulation results show that PRNet can learn a low-PAPR, decodable constellation mapping without side information. The paper reports a dramatic PRNet CCDF shift; our code audit indicates that the published red curve corresponds to a peak-amplitude proxy, while standard power-PAPR evaluation still shows a meaningful but less extreme reduction. The two-stage training procedure identifies $\eta^\star = -15$ dB as the best corruption level in the paper's setting, and $\lambda = 0.01$ as its selected BER–PAPR trade-off. Computationally, the single forward pass is faster than the multi-IFFT search of SLM/PTS when GPU acceleration is used.

2.7 Common Misconceptions

Common Misconception

"Once trained, PRNet works for any N and modulation."
The encoder and decoder dimensions are fixed by the chosen representation. In the paper's one-hot formulation, the network input/output dimension is NM and the encoder's physical signal output is $2N$; in our coordinate-based reproduction, the network input/output dimension is $2N$. Either way, a PRNet trained for $N = 64$ subcarriers with QPSK cannot be directly used for $N = 128$ or for 16-QAM, because the weight matrices and learned signal statistics no longer match. Changing N or M requires a new architecture or at least substantial retraining.

Common Misconception

"The AWGN-trained model will work on a Rayleigh channel."

The decoder learns an inverse mapping that is specific to the channel statistics it was trained on. If the model is trained with AWGN noise only and then deployed on a Rayleigh fading channel — where the signal is randomly attenuated and rotated by complex fading coefficients — the decoder will fail because it has never encountered that kind of distortion. The constellation shapes that are "safe" under AWGN may not be safe under fading, and the decoder's decision boundaries are calibrated for a noise distribution it has never seen. The CCDF may still look similar because PAPR is measured before the channel, but the BER can degrade sharply. Retraining or fine-tuning under the target channel model is required when the channel statistics change.

Common Misconception

"PRNet learns QPSK symbols and rotates them."

PRNet does not start with QPSK points and apply phase rotations (that would be SLM). Instead, the encoder takes the raw bit sequence and directly computes entirely new complex numbers from scratch. These numbers are not constrained to any standard constellation grid — they are continuous-valued coordinates in the complex plane, shaped specifically for the current bit pattern. There are no "base QPSK points" being modified; the encoder invents its own constellation geometry from the ground up [1].

2.8 Limitations and Open Problems

PRNet is a foundational result, but several limitations remain (from the paper's discussion and our own implementation experience):

1. **Scalability to large N:** Fully connected layers scale poorly as N grows. The first encoder FC layer has $d_{\text{in}} \times d_{\text{out}} = d_{\text{in}} \times 2048$ weights, where $d_{\text{in}} = NM$ for the paper's one-hot representation or $d_{\text{in}} = 2N$ for the coordinate representation used in our code. With the paper's $N = 64$, $M = 4$ setup, the first layer alone has $256 \times 2048 = 524{,}288$ weights. Scaling this dense design to practical large-FFT systems quickly becomes expensive, so convolutional, unfolding, or attention-based structures are more plausible for larger N [1].
2. **Channel mismatch:** The paper models a Rayleigh fading channel, and the network's performance is tightly coupled to how well the training channel matches the actual deployment environment. Extending PRNet to clearly

specified frequency-selective channels, imperfect channel state information (CSI), pilot overhead, and rapidly time-varying scenarios remains an open problem [1].

3. **Retraining across configurations:** Any change in system parameters — modulation order (M), number of subcarriers (N), pilot structure, coding rate — changes the dimensionality or statistical properties of the signal. If the dimensions change, the architecture must change as well; if only the channel or operating point changes, fine-tuning may be possible, but a fresh validation campaign is still required.

4. **PAPR metric ambiguity:** The paper defines CCDF in terms of PAPR, but the released PRNet code plots a peak-amplitude proxy for the PRNet CCDF. This does not invalidate the learned approach, but it means the most dramatic Fig. 3 number should be interpreted carefully and standard power-PAPR results should be reported separately.

5. **Interpretability:** PRNet learns an implicit constellation shaping rule that is encoded in millions of weight values. Understanding *what* specific shaping the network has learned — for instance, whether it has discovered a structure analogous to a known coding scheme — is extremely difficult. This makes it hard to diagnose failures or to prove guarantees about the system's worst-case behaviour, unlike explicit methods like SLM where the phase rotation rule is transparent.

2.9 Connection to Other Approaches

PRNet established the autoencoder paradigm for PAPR reduction. Subsequent works build on Kim's core insight — treat the communication chain as an end-to-end trainable system and include PAPR explicitly in the learning objective — while addressing one or more of PRNet's limitations:

- **Hao et al. (2019)** explicitly connect the autoencoder to the SLM framework by constraining the encoder to learn *phase factors* rather than an unconstrained nonlinear mapping. This retains SLM's interpretability while gaining the autoencoder's ability to optimise phase sequences via gradient descent.
- **Alnaseri et al. (2025)** transfer the autoencoder idea to coherent optical OFDM (CO-OFDM), incorporating fibre-channel effects such as chromatic dispersion and Kerr nonlinearity into the end-to-end training pipeline.

- **Zou et al. (2021)** use a related DNN architecture within a signal-cancellation framework rather than a pure autoencoder, adding a dedicated peak-cancellation and signal-recovery module that achieves PAPR reduction without requiring a jointly trained decoder.

In short, PRNet was the first to demonstrate that deep learning can jointly optimise BER and PAPR in OFDM. Its limitations — scalability, channel sensitivity, and interpretability — define the natural research agenda that subsequent works address.

APPENDIX A

PRNet Simulation Parameters

This appendix consolidates all simulation and system parameters used in the PRNet experiments, as reported in Kim et al. (2017) [1].

TABLE A.1: System and training parameters for PRNet [1].

Parameter	**Value**
OFDM System	
Number of subcarriers (N)	64
Modulation	QPSK
Oversampling factor (L)	4
Channel (training)	AWGN / Rayleigh flat fading
Network Architecture	
Hidden layers per DNN (encoder/decoder)	5
Neurons per hidden layer	2048
Hidden activation	ReLU
Output activation	Linear
Batch Normalization	Yes (after every FC layer)
Training Configuration	
Optimizer	Adam
Learning rate	10^{-4}
Batch size	400
Training steps	500,000
Optimal corruption level ($\eta^{\star}$)	-15 dB
PAPR loss weight (λ)	0.01
Reported Results	
PAPR reduction at CCDF $= 10^{-3}$	$\approx$ 6–7 dB
Inference speedup vs. PTS (GPU)	$\approx 6\times$

Key Insight

The two key hyperparameters that define PRNet's operating point are:

- $\eta^\star = -15$ dB — the optimal training corruption level, identified in Stage 1 by sweeping $\eta \in \{0, -5, -10, -15, -20, -25, -30\}$ dB.
- $\lambda = 0.01$ — the PAPR loss weight, which makes reconstruction $100\times$ more important than PAPR reduction in the joint loss, preserving BER while still achieving significant PAPR gains.

APPENDIX B

Glossary of Deep Learning Terms

This appendix provides concise definitions of deep learning concepts referenced throughout the document, intended as a quick reference for readers with a communications background.

Neuron
: The basic computational unit of a neural network. It computes a weighted sum of its inputs, adds a bias, and applies an activation function: $y = f(\boldsymbol{w}^T \boldsymbol{x} + b)$.

Weight
: A learnable parameter in a neural network that scales the connection between two neurons. The collection of all weights (and biases) constitutes the model's trainable parameters, denoted $\boldsymbol{\theta}$.

Bias
: A learnable constant added to the weighted sum in a neuron before the activation function is applied: $z = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$. It allows the network to shift the activation function, giving it more flexibility.

Activation Function
: A nonlinear function applied element-wise to the output of a neural network layer. Common choices include ReLU, sigmoid, tanh, softmax, and the linear (identity) function. Without nonlinear activations, a multi-layer network would collapse to a single linear transformation.

Fully Connected (FC) Layer
: A layer where every neuron is connected to every neuron in the previous layer via a weight matrix $\boldsymbol{W}$. Also called a dense layer. The computational cost of an FC layer scales as $\mathcal{O}(n_{\text{in}} \times n_{\text{out}})$.

Forward Pass
: The computation that propagates input data through the network, layer by layer, to produce an output. During inference (deployment), only the forward pass is needed—backpropagation is not required.

Inference
: The process of using a trained model to make predictions on new, unseen data.

In the PAPR context, inference means performing one forward pass per OFDM symbol at real-time rates.

Loss Function

A scalar function that quantifies how far the network's output is from the desired output. Training minimises this function. Common choices include MSE for regression and cross-entropy for classification. In PAPR, custom losses combine PAPR and MSE terms.

Training

The iterative process of adjusting network parameters to minimise the loss function. Involves repeated forward passes, loss computation, backpropagation, and parameter updates over many epochs.

Gradient

The vector of partial derivatives of the loss function with respect to each trainable parameter. It indicates the direction and magnitude of the steepest increase in loss; the optimizer updates parameters in the opposite direction.

Backpropagation

The algorithm used to compute gradients of the loss function with respect to all network parameters by applying the chain rule of calculus, layer by layer, from the output back to the input. These gradients are then used by the optimizer to update the weights.

Optimizer

The algorithm that updates network parameters based on computed gradients. SGD (Stochastic Gradient Descent) is the simplest; Adam (Adaptive Moment Estimation) is the most commonly used in practice, combining momentum and per-parameter learning rate adaptation.

Learning Rate (η)

A hyperparameter that controls the step size of each parameter update. Too large a value causes training to diverge; too small a value makes convergence extremely slow.

Epoch

One complete pass through the entire training dataset. Training typically requires hundreds of epochs before the network converges to a good solution.

Batch

A subset of the training data processed together in one forward and backward pass. Using batches (rather than the entire dataset) enables efficient GPU utilisation and introduces beneficial stochastic noise into training.

Convergence

The point during training at which the loss function stabilises and further epochs yield negligible improvement. A model that fails to converge may be underfitting (too simple) or have a learning rate that is too high.

Hyperparameter

A configuration value set before training begins and not learned from data. Examples include learning rate, batch size, number of layers, number of epochs, and the loss balance factor α.

Overfitting

When a network learns to memorise the training data rather than generalise to unseen data. Symptoms include very low training loss but high validation/test loss. Mitigated by dropout, regularisation, and early stopping.

Regularisation

Any technique that prevents overfitting by constraining the model's complexity. Examples include dropout, weight decay (L_2 regularisation), and batch normalization.

Dropout

A regularisation technique that randomly sets a fraction of neuron outputs to zero during each training step. This prevents neurons from co-adapting and encourages the network to learn redundant representations, reducing overfitting.

Batch Normalization (BatchNorm)

A technique that normalises the inputs to each layer by subtracting the batch mean and dividing by the batch standard deviation. This stabilises training, allows higher learning rates, and acts as a mild regulariser.

Vanishing Gradients

A problem where gradients become extremely small as they propagate through many layers during backpropagation, causing early layers to learn very slowly or not at all. ReLU activations and residual connections help mitigate this.

Autoencoder

A neural network architecture consisting of an encoder that maps input data to a lower-dimensional (or transformed) representation, and a decoder that reconstructs the original data. In the PAPR context, the encoder produces a low-PAPR signal and the decoder recovers the original data symbols.

References

[1] M. Kim, W. Lee, and D.-H. Cho, “A novel papr reduction scheme for ofdm system based on deep learning,” *IEEE Communications Letters*, vol. 22, no. 3, pp. 510–513, 2017.